

CEG 4799 A00 / CSI 4539 A00

Conception de systèmes informatiques sécuritaires

Laboratoire 2 - Automne 2026 | Énoncé et grille d'évaluation

Attaques mémoire, outils de bas niveau et première défense par le typage

Énoncé version 1.1 — 3 octobre 2026. Cet énoncé complète le plan de cours (version 1.1); toute modification sera annoncée sur Brightspace.

Professeur	Mohamed Ali Ibrahim, ing., Ph.D.
Bureau / courriel	STE 4068 — mibrahi3@uottawa.ca
Heures de bureau	Par rendez-vous
Assistant à l'enseignement	Mohamed Nefsi — mnefs047@uottawa.ca
Période du cours	Du 9 septembre au 9 décembre 2026
Site du cours	Brightspace — https://uottawa.brightspace.com/

Remise du rapport : vendredi 16 octobre 2026, 23 h 59, sur Brightspace.

Démonstration devant l'assistant : mardi 13 octobre 2026 (séance 2), présence de tous les membres obligatoire.

Présence obligatoire à la démonstration. La démonstration se fait devant l'assistant à la fin de la séance 2. La présence de tous les membres de l'équipe est obligatoire; chaque membre doit pouvoir expliquer et reproduire n'importe quelle partie du travail remis. Un membre absent sans justificatif accepté par le professeur reçoit zéro pour la partie « Démonstration » (section 8).

Usage autorisé. Les techniques offensives de ce laboratoire (débordement de tampon, shellcode, return-to-libc) ne s'appliquent qu'aux programmes fournis dans l'environnement isolé du cours (machine virtuelle ou conteneurs SEED). Toute action contre un système tiers, y compris un système de l'Université, est interdite et traitée comme une inconduite (plan de cours, « Usage autorisé »). L'entente d'usage autorisé signée au laboratoire 1 demeure en vigueur.

Renseignements sur le laboratoire

Élément	Détail
Séance 1 — shellcode et débordement	Mardi 6 octobre, 14 h 30 – 17 h 20. Disposition de la pile, développement du shellcode, débordement de tampon (Niveaux 1 à 3) et contre-mesures (dash, ASLR, StackGuard, pile non exécutable).
Séance 2 — return-to-libc, défense et démonstration	Mardi 13 octobre, 14 h 30 – 17 h 20. Attaque return-to-libc, chaîne source–assembleur–code machine, réécriture en Ada et démonstration devant l’assistant.
Remise du rapport	Vendredi 16 octobre 2026, 23 h 59, sur Brightspace.
Pondération	5 % de la note finale; grille sur 100 points (section 10).
Équipe	Deux étudiants; même équipe qu’au laboratoire 1, sauf indication contraire.
Environnement de référence	Machine virtuelle SEED Ubuntu 20.04 (suite SEED 2.0); les conteneurs Docker sont autorisés si la même configuration y a été testée (requis pour l’extension Serveur). C et assembleur x86; gcc, gdb, objdump, readelf, nasm, ld, execstack, sysctl. Pour la partie D : chaîne GNAT FSF et gprbuild (installés par Alire à la séance 0) et extension Ada de Visual Studio Code.
Thèmes SEED (fournis)	« Shellcode Development Lab », « Buffer Overflow Attack Lab (Set-UID Version) » et « Return-to-libc Attack Lab ». Extension obligatoire : « Buffer Overflow Attack Lab (Server Version) ». Les dossiers « Labsetup » correspondants sont fournis dans le dépôt.
Préparation	Manuel de Wenliang Du, ch. 4 (débordement de tampon), 5 (return-to-libc) et 9 (shellcode). Introduction to Ada, ch. « Strongly typed language », « Records » et « Arrays ». Notes des séances 4 à 10.
Composante CLO	L2-B — relier source C, assembleur et code machine et analyser l’effet des options de compilation (CLO 9, PI 5.1, GA 5, niveau Advanced-Applied).
Légué au projet final	Un corpus d’entrées mémoire (débordement, return-to-libc) et les propriétés violées écrites comme assertions vérifiables.

1. Contexte et objectifs

Les cinq laboratoires du cours sont cumulatifs et convergent vers le projet final : chacun produit des modules, des propriétés de sécurité et un corpus d’entrées dangereuses réutilisés au laboratoire suivant. Le laboratoire 2 étudie la deuxième grande classe de vulnérabilités du cours : les erreurs de gestion de la mémoire des programmes écrits en C. L’équipe reproduit, dans l’environnement isolé du cours, un débordement de tampon qui détourne le flot de contrôle, développe et teste un shellcode, puis monte une attaque return-to-libc qui réutilise la bibliothèque C pour contourner la pile non exécutable. Elle active et désactive ensuite les contre-mesures — protection de dash, ASLR, StackGuard et pile non exécutable — pour mesurer ce que chacune bloque réellement, relie le code source à l’assembleur et au code machine, et termine par la première défense apportée par le langage : la réécriture en Ada d’une routine d’analyse d’entrée, où le contrôle de contrainte transforme le débordement en exception maîtrisée.

La démarche du cours s’applique intégralement ici, en quatre temps : observer l’attaque, remonter à la cause, écrire l’hypothèse violée comme propriété explicite, puis vérifier que le code corrigé la respecte. Un débordement de tampon ou une réutilisation de code comme le return-to-libc ne trahit pas un manque d’effort : chacun exploite une hypothèse que le programme tenait pour acquise sans l’écrire — sur la longueur d’une entrée, sur une borne d’index,

sur l'intégrité de l'adresse de retour. Chaque hypothèse implicite devient ici une exigence de sécurité vérifiable, qui préfigure les contrats prouvés en SPARK aux laboratoires 3 à 5.

À la fin de ce laboratoire, l'étudiant est capable de :

- expliquer la disposition de la pile d'exécution et du cadre d'appel, et localiser le tampon, le pointeur de cadre sauvegardé et l'adresse de retour;
- écrire et tester un shellcode fonctionnel, de l'assembleur au code machine, et éliminer les octets nuls;
- construire une entrée malveillante qui écrase l'adresse de retour et exécute le code injecté avec le privilège du programme;
- activer et désactiver la protection de dash, l'ASLR, StackGuard et la pile non exécutable, et documenter ce que chaque contre-mesure bloque et ne bloque pas;
- monter une attaque return-to-libc qui contourne la pile non exécutable en réutilisant `system()` et la chaîne « `/bin/sh` » de la bibliothèque C;
- relier le code source C à l'assembleur et au code machine, et analyser l'effet des options de compilation et des contre-mesures (composante L2-B, CLO 9);
- réécrire la routine d'analyse d'entrée vulnérable en Ada avec sous-types contraints, et observer le contrôle de contrainte transformer le débordement en exception maîtrisée;
- écrire les hypothèses violées comme assertions vérifiables et constituer un corpus d'entrées mémoire documenté, légué au projet final.

Lien avec les résultats d'apprentissage (CLO) : le laboratoire 2 porte la composante L2-B (CLO 9, PI 5.1, GA 5, niveau Advanced-Applied) : choisir et utiliser compilateur, désassembleur et débogueur, relier source C, assembleur et code machine, analyser l'effet d'une option de compilation ou d'une contre-mesure et en exposer les limites. Il alimente aussi le projet final (laboratoire 5) : le corpus d'entrées mémoire est rejoué par le banc d'essai du moniteur, et les propriétés violées sont réécrites en assertions vérifiables au laboratoire 3.

2. Plateforme de travail et préparation de l'environnement

Rappel. L'environnement SEED (suite SEED 2.0) a été installé et vérifié au laboratoire 1. Le laboratoire 2 réutilise le même environnement et le même dépôt Git. La machine virtuelle SEED Ubuntu 20.04 est l'environnement de référence; Docker et Docker Compose sont requis pour l'extension Serveur obligatoire (section 3, E9).

Les fichiers de ce laboratoire proviennent des thèmes « Shellcode Development Lab », « Buffer Overflow Attack Lab (Set-UID Version) » et « Return-to-libc Attack Lab » de seedsecuritylabs.org; l'extension obligatoire utilise « Buffer Overflow Attack Lab (Server Version) ». Les dossiers « Labsetup » correspondants sont fournis dans le dépôt. Les outils sont déjà présents dans l'environnement SEED : `gcc`, `gdb`, `objdump`, `readelf`, `nasm`, `ld`, `execstack` et `sysctl`. Pour la partie D, la chaîne GNAT FSF et `gprbuild` sont installés par Alire (séance 0), avec l'extension Ada de Visual Studio Code.

2.1 Vérification des outils

Au début de la séance 1, ouvrir un terminal dans l'environnement SEED et confirmer que la chaîne d'outils de bas niveau est prête :

Commande	Ce qu'elle confirme
<code>gcc --version</code> , <code>gdb --version</code> , <code>objdump --version</code>	Le compilateur, le débogueur et le désassembleur sont présents.
<code>nasm -v</code> et <code>ld --version</code>	L'assembleur et l'éditeur de liens sont présents (développement de shellcode).

Commande	Ce qu'elle confirme
<code>gcc -m32 prog.c -o prog</code>	La compilation 32 bits fonctionne (bibliothèques multilib installées).
<code>sysctl kernel.randomize_va_space</code>	L'état de l'ASLR est lisible (0, 1 ou 2) et modifiable par <code>sudo sysctl -w</code> .
<code>gcc -fno-stack-protector -z execstack prog.c -o prog</code>	Les options de contre-mesure sont acceptées par le compilateur.
<code>gnatmake --version (ou alr --version)</code>	La chaîne Ada/GNAT est disponible pour la partie D.

2.2 Contrôle des contre-mesures

Chaque essai d'attaque indique explicitement l'état des contre-mesures. On les règle dans l'environnement SEED isolé comme suit :

Contre-mesure	Activer / désactiver
Protection de dash	<code>sudo ln -sf /bin/zsh /bin/sh</code> (contournée) ou <code>sudo ln -sf /bin/dash /bin/sh</code> (active, à déjouer par <code>setuid(0)</code> dans le shellcode).
ASLR	<code>sudo sysctl -w kernel.randomize_va_space=0</code> (désactivé) ou <code>=2</code> (actif).
StackGuard (canari de pile)	<code>gcc -fno-stack-protector</code> (désactivé) ou compilation sans cette option (actif).
Pile non exécutable (NX)	<code>gcc -z noexecstack</code> (active la protection) ou <code>-z execstack</code> (pile exécutable).

Ces réglages ne s'appliquent qu'à l'environnement SEED isolé; ils ne doivent jamais être modifiés sur un poste connecté au réseau de l'Université.

2.3 Configuration de référence des protections par essai

Pour rendre les essais reproductibles, la configuration de départ attendue pour les principales exploitations est la suivante. La désactivation de l'ASLR simplifie la mise au point; une variante avec ASLR actif (force brute sur 32 bits) est explorée dans l'essai E4.

Essai	ASLR	Pile (NX)	StackGuard et compilation
E3 Débordement et shellcode (T3)	Désactivé	Désactivée	Sans canari; <code>gcc -m32 -fno-stack-protector -z execstack</code> (stack.c, binaire Set-UID root).
E4 Contre-mesures (T4)	Varie	Varie	On part de la configuration d'E3, puis on réactive chaque protection à tour de rôle (dash, <code>randomize_va_space=2</code> , <code>-fstack-protector</code> , <code>-z noexecstack</code>) pour en observer l'effet.
E5 Return-to-libc (T5)	Désactivé	Activée	Sans canari; <code>gcc -m32 -fno-stack-protector -z noexecstack</code> (retlib.c, binaire Set-UID root).

2.4 Isolement et sécurité

- Toutes les manipulations et attaques restent dans l'environnement SEED isolé. Aucune attaque n'est dirigée vers un système extérieur, y compris un système de l'Université (plan de cours, « Usage autorisé »).
- Les binaires vulnérables Set-UID sont compilés localement dans l'environnement et ne sont jamais installés ailleurs; les conteneurs de l'extension Serveur restent sur un réseau Docker isolé.
- Les scripts d'attaque et les shellcodes restent dans le dépôt de l'équipe et ne sont ni diffusés ni exécutés hors de l'environnement du cours.

3. Travail exigé

Le travail est organisé en cinq parties. Chaque exigence porte un identifiant (E1 à E9) repris dans le tableau des essais (section 6) et dans la grille d'évaluation (section 10). La partie E (E9, version Serveur) est une extension obligatoire : elle rejoue l'attaque dans un contexte d'exploitation à distance et est notée aux parties A et B.

Partie A — Shellcode et débordement de tampon (version Set-UID)

Id	Exigence	Critère d'acceptation
E1	Disposition de la pile et cadre d'appel	Compiler le programme vulnérable stack.c en binaire Set-UID root 32 bits (make : gcc -m32 -z execstack -fno-stack-protector, puis chown root et chmod 4755). Sous gdb (stack-L1-dbg : b bof, next, p \$ebp, p &buffer), localiser le tampon, le pointeur de cadre sauvegardé et l'adresse de retour, et calculer la distance exacte entre le tampon et l'adresse de retour. Documenter la disposition de la pile (schéma).
E2	Développement du shellcode	Assembler un shellcode 32 et 64 bits (nasm, ld) à partir des fichiers fournis (hello.s, mysh64.s, another_sh64.s), en extraire le code machine (objdump -Intel -d, xxd, convert.py) et éliminer tous les octets nuls. Vérifier le shellcode de façon autonome avec call_shellcode.c (a32.out, a64.out). Expliquer les deux approches (chaîne dans le segment de code via call; construction sur la pile).
E3	Exploitation du débordement (Niveaux 1 à 3)	Compléter exploit.py (shellcode, start, ret, offset) pour produire badfile et obtenir un interpréteur root sur stack-L1 (Niveau 1). Réaliser le Niveau 2 (taille du tampon inconnue : une seule charge valable pour la plage 100–200 octets, sans force brute) et le Niveau 3 (binaire 64 bits : gérer les octets nuls de l'adresse). Le Niveau 4 (tampon de 10 octets) est en bonus. Justifier chaque valeur d'exploit.py.

Partie B — Contre-mesures et return-to-libc

Id	Exigence	Critère d'acceptation
E4	Contre-mesures et leurs limites	Montrer, pour chaque contre-mesure, ce qu'elle bloque, ses limites et le contournement prévu : déjouer la protection de dash en ajoutant setuid(0) au shellcode (lien /bin/sh → /bin/dash); déjouer l'ASLR par force brute (randomize_va_space=2, brute-force.sh) sur stack-L1; réactiver StackGuard (compilation sans -fno-stack-protector) et la pile non exécutable (-z noexecstack), et expliquer pourquoi l'attaque de la partie A échoue alors.
E5	Return-to-libc	Sur le programme retlib.c (compilé -m32 -fno-stack-protector -z noexecstack, Set-UID root), contourner la pile non exécutable sans injecter de code. Trouver les adresses de system() et de exit() (gdb : p system, p exit) et l'adresse de la chaîne « /bin/sh » (variable d'environnement exportée, lue par un petit programme prtenv). Compléter exploit.py (offsets X, Y, Z) pour que bof() retourne vers system("/bin/sh"). Traiter les deux variations : exit() est-il nécessaire; effet d'un changement du nom du binaire sur l'adresse.

Partie C — Outils de bas niveau (composante L2-B)

Id	Exigence	Critère d'acceptation
E6	Chaîne source–assembleur–code machine	Pour la fonction vulnérable et pour le shellcode (E2), relier le code source (C ou assembleur), le désassemblage (objdump -Intel -d, xxd) et le code machine; identifier le prologue, l'épilogue et l'instruction ret. Analyser et documenter l'effet d'au moins deux options de compilation ou contre-mesures (p. ex. -fstack-protector, -O2, -z execstack / -z noexecstack, -m32) sur l'assembleur produit et sur la faisabilité de l'attaque, et en exposer les limites. Composante L2-B (CLO 9).

Partie D — Première défense par le typage (Ada)

Id	Exigence	Critère d'acceptation
E7	Réécriture en Ada	Réécrire la routine d'analyse d'entrée vulnérable (copie dans un tampon de taille fixe) en Ada, en utilisant des sous-types contraints (chaîne de longueur bornée, index contraint). Rejouer la même entrée qu'en E3 et montrer que le contrôle de contrainte lève un Constraint_Error maîtrisé au lieu de déborder; traiter l'exception dans un état sûr. De façon générale, montrer que l'entrée est rejetée sans corruption mémoire, par contrôle de contrainte et traitement sûr de l'exception.
E8	Propriétés vérifiables et corpus légué	Écrire les hypothèses violées comme assertions ou propriétés explicites (domaine de longueur admissible, borne d'index, invariant du tampon), formulées pour pouvoir être vérifiées et préfigurer les contrats SPARK. Constituer le corpus d'au moins huit entrées mémoire dangereuses documentées (débordement, return-to-libc), légué au projet final.

Partie E — Extension obligatoire : exploitation à distance (version Serveur)

Id	Exigence	Critère d'acceptation
E9	Exploitation à distance (version Serveur)	Rejouer l'attaque sur la version Serveur en conteneur : compiler et installer le serveur (make, make install; binaire 32 bits -static), démarrer les conteneurs (dcbuild, dcpu), puis exploiter stack.c à distance par la connexion TCP (port 9090) à l'aide d'exploit.py côté client. Documenter les différences avec l'attaque locale (absence d'accès au binaire, adresses, shell inversé).

4. Exigences techniques et de codage

- **Environnement isolé.** Toutes les manipulations et attaques se font dans l'environnement SEED (machine virtuelle ou conteneur). Aucun système tiers n'est visé (plan de cours, « Usage autorisé »).
- **Reproductibilité.** Chaque exploitation est fournie comme script ou commande jouable, accompagnée de la trace obtenue; l'état de l'ASLR et les options de compilation sont explicitement indiqués pour chaque essai. Un lecteur doit pouvoir reproduire le résultat à partir du dépôt.
- **Code vulnérable et code de défense.** Les programmes vulnérables fournis (stack.c, retlib.c) ne sont pas modifiés pour les attaques (toute recompilation avec des options différentes est documentée); la routine de défense en Ada compile et s'exécute telle quelle dans l'environnement.
- **Hypothèses écrites.** Chaque hypothèse de sécurité (longueur d'entrée, borne d'index, intégrité de l'adresse de retour) est écrite explicitement (commentaire d'assertion, précondition, domaine de valeurs admissibles), de manière à pouvoir être vérifiée. Ces propriétés préparent les contrats prouvés en SPARK aux laboratoires 3 à 5.
- **Outils d'IA générative.** Permis comme outil d'appoint, à condition de le déclarer dans le rapport (outil, version, usage) et de pouvoir expliquer et justifier tout code, script ou texte remis. Interdits pendant la démonstration et la vérification individuelle.

Organisation du projet et dépôt Git

- **Dépôt d'équipe.** Le même dépôt Git qu'au laboratoire 1, enrichi des scripts d'attaque mémoire, du shellcode, de la routine de défense en Ada, des traces et du fichier JOURNAL.md; chaque membre valide ses propres modifications sous son nom.
- **Rotation des rôles.** Les rôles réalisation/validation sont inversés par rapport au laboratoire 1 : à chaque séance, un membre réalise et l'autre valide (essais, traces, relecture); l'inversion est consignée dans le journal.
- **Corpus.** Le corpus d'entrées mémoire est rangé dans un dossier dédié du dépôt (une entrée par fichier ou par ligne, avec sa description), prêt à être rejoué au projet final.

5. Déroulement des séances

Séance	Activités	Résultat attendu à la fin de la séance
Séance 1 6 oct.	Disposition de la pile et cadre d'appel (E1); développement et test du shellcode (E2); exploitation du débordement, Niveaux 1 à 3 (E3); contre-mesures : dash, ASLR, StackGuard, pile non exécutable (E4). Amorcer les relevés objdump pour l'analyse source–assembleur–code machine (E6).	E1 à E4 réalisés; shellcode sans octet nul; exploit du débordement rejouable; effet des contre-mesures observé; relevés d'E6 amorcés.
Entre les séances 6 au 13 oct.	Finaliser hors séance l'analyse source–assembleur–code machine et l'effet des options de compilation (E6), à partir des relevés de la séance 1.	Analyse L2-B rédigée et prête à être vérifiée sur traces à la séance 2.
Séance 2 13 oct.	Attaque return-to-libc sur retlib.c (E5); exploitation à distance de la version Serveur en conteneur (E9); vérification de l'analyse L2-B (E6); réécriture en Ada et contrôle de contrainte (E7); propriétés et corpus (E8); démonstration devant l'assistant. La construction des conteneurs Serveur peut être préparée entre les séances.	Toutes les exigences démontrées, y compris l'exploitation à distance; Constraint_Error observé à la place du débordement; corpus constitué; démonstration validée par l'assistant en présence de tous les membres.
Hors séance au plus tard le 16 oct.	Finalisation du rapport, tableau des essais avec traces, note sur les contre-mesures et leurs limites, déclaration d'IA générative; remise sur Brightspace.	Archive remise avant 23 h 59 le vendredi 16 octobre 2026.

6. Essais et traces exigés

Le rapport présente un tableau des essais reprenant au minimum les cas ci-dessous, avec pour chacun le résultat observé et une preuve (trace de terminal, capture d'écran, extrait de journal). Les traces montrent la commande, l'état des contre-mesures (ASLR et options de compilation), l'identité effective (id, getresuid) et l'effet obtenu.

Essai	Procédure	Résultat attendu	Preuve
T1 Disposition de la pile	Localiser le tampon, le pointeur de cadre sauvegardé et l'adresse de retour; calculer la distance (E1).	Distance exacte obtenue; disposition de la pile documentée.	Trace gdb/objdump et schéma.
T2 Shellcode	Assembler et exécuter le shellcode de façon autonome (E2).	/bin/sh obtenu (a32.out, a64.out); aucun octet nul.	Fichiers .s commentés, objdump et trace.

Essai	Procédure	Résultat attendu	Preuve
T3 Débordement (Niveaux 1–3)	Produire badfile avec exploit.py et attaquer stack-L1, puis Niveaux 2 et 3 (E3).	Interpréteur root obtenu; une seule charge pour la plage (N2); 64 bits (N3).	Scripts rejouables et traces id/getresuid.
T4 Contre-mesures	Déjouer dash, forcer l'ASLR, réactiver StackGuard et la pile non exécutable (E4).	L'effet et les limites de chaque contre-mesure sont montrés.	Traces avant/après, valeurs sysctl et options gcc.
T5 Return-to-libc	Compléter exploit.py (X, Y, Z) et attaquer retlib.c (E5).	system("/bin/sh") exécuté sans code injecté, pile non exécutable active.	Script rejouable, adresses, trace.
T6 Source–asm–machine (L2-B)	Relier C/assembleur, désassemblage et code machine; comparer deux options (E6).	Prologue, épilogue et ret identifiés; effet des options documenté.	Extraits objdump annotés, tableau comparatif.
T7 Défense Ada	Rejouer l'entrée de T3 sur la routine écrite en Ada (E7).	Un Constraint_Error est levé et traité; aucun débordement.	Source Ada et trace d'exécution.
T8 Exploitation à distance (Serveur)	Exploiter stack.c à distance par TCP sur le serveur en conteneur (E9).	Interpréteur ou shell inversé obtenu à distance (port 9090).	exploit.py côté client et trace du conteneur.

E8 est une exigence documentaire (propriétés violées écrites comme assertions et corpus d'entrées mémoire); elle n'a pas d'essai distinct et est évaluée notamment par les critères C2 et D2. L'essai T8 correspond à l'extension obligatoire E9 (version Serveur).

7. Livrables

Une seule archive par équipe, nommée CEG4799_Lab2_<NomA>_<NomB>.zip, remise sur Brightspace au plus tard le vendredi 16 octobre 2026 à 23 h 59. Tout travail remis après l'échéance reçoit la note de zéro, sauf prolongation autorisée par le professeur ou accommodement approuvé.

Livrable	Contenu	Format
1. Code et scripts	Programme stack.c (recompilation documentée) et exploit.py (E3); shellcode en assembleur et son code machine (hello.s, mysh64.s, another_sh64.s, convert.py, call_shellcode.c) (E2); retlib.c et son exploit.py (E5); routine de défense en Ada et son fichier de projet (.gpr) (E7). Le tout doit compiler et s'exécuter dans l'environnement SEED.	Dossier dans l'archive
2. Rapport	Structure de la section 7.1. Rapport rédigé en français, langue du cours.	PDF
3. Composante L2-B	Document reliant source C/assembleur, désassemblage et code machine, avec l'analyse de l'effet des options de compilation et des contre-mesures et ses limites (E6).	Section du rapport ou dossier dédié
4. Journal d'équipe	Rôles et rotation, journal des décisions et des essais (JOURNAL.md); contributions substantielles et identifiables des deux membres dans l'historique Git.	Markdown + adresse du dépôt

Livrable	Contenu	Format
5. Corpus d'entrées mémoire	Au moins huit entrées documentées (débordement, return-to-libc), une par fichier ou par ligne avec sa description; légué au projet final.	Dossier corpus/ dans l'archive
6. Extension Serveur (obligatoire)	Fichiers (server-code, attack-code, docker-compose.yml) et traces de l'exploitation à distance en conteneur (E9, T8).	Dossier dans l'archive
7. Traces	Traces de terminal et captures des essais T1 à T8, montrant commande, état des contre-mesures, identité effective et effet.	Annexe du rapport ou dossier traces/
8. Déclaration	Sources utilisées et déclaration d'usage d'outils d'IA générative (outil, version, usage), ou mention « aucun usage ».	Section du rapport

7.1 Structure du rapport

1. Introduction et modèle de menace : objectif, description des programmes vulnérables, actifs protégés et surfaces d'entrée en mémoire.
2. Disposition de la pile et contre-mesures : cadre d'appel et distance au tampon (E1); effet de la protection de dash, de l'ASLR, de StackGuard et de la pile non exécutable (E4).
3. Exploitations mémoire : shellcode (E2), débordement de tampon Niveaux 1 à 3 (E3) et return-to-libc (E5); pour chacune, sa cause première, la trace obtenue et la propriété violée écrite explicitement.
4. Outils de bas niveau (L2-B) : chaîne source–assembleur–code machine et effet des options de compilation (E6), avec ses limites.
5. Défense par le typage : réécriture en Ada et contrôle de contrainte (E7); propriétés corrigées écrites comme assertions (E8); essai T7.
6. Corpus légué et limites : contenu du corpus, ce que le typage et les contre-mesures garantissent et ce qu'ils ne garantissent pas.
7. Extension à distance (version Serveur) : exploitation en conteneur par TCP (E9) et différences avec l'attaque locale.
8. Travail d'équipe : rôles par séance, décisions, désaccords éventuels et leur traitement, contribution de chacun (référence à l'historique Git).
9. Déclaration des sources et de l'usage d'outils d'IA générative; annexes (code commenté, traces).

8. Démonstration

Présence de tous les membres obligatoire. La démonstration a lieu devant l'assistant pendant la séance 2 (mardi 13 octobre, section A01), sur l'environnement SEED de l'équipe, à partir du dépôt Git. Tous les membres de l'équipe doivent être présents pendant toute la démonstration.

- **Déroulement.** L'équipe dispose de 20 minutes : 15 minutes d'essais, puis 5 minutes de questions individuelles. Sont démontrés en direct les essais T3, T5, T8 et T7 (débordement, return-to-libc, exploitation à distance, défense Ada); les essais T1, T2, T4 et T6 sont vérifiés par l'assistant à partir des traces et des scripts du dépôt. L'assistant note chaque critère de la partie A. L'environnement doit être prêt, les conteneurs Serveur démarrés, l'ASLR et les options de compilation réglés, et les scripts rejouables à l'heure de passage.
- **Questions individuelles.** L'assistant pose à chaque membre au moins deux questions sur une partie tirée au sort du travail (disposition de la pile, shellcode, return-to-libc, exploitation à distance, contrôle de contrainte en Ada) et peut demander une petite modification ou une exploitation en direct. La note de la partie A est individualisée selon les réponses (critère A6); un membre incapable d'expliquer le travail remis voit sa note de démonstration réduite, et le cas est traité selon le règlement A-4 si le travail n'est manifestement pas le sien.

- **Absence.** Avec un justificatif accepté par le professeur, la démonstration est reprise sur rendez-vous avec l'assistant dans les cinq jours ouvrables. Sans justificatif, le membre absent reçoit zéro pour la partie A (30 points); le membre présent est évalué normalement. La remise du rapport reste due à la date prévue.
- **Démonstration incomplète.** Une équipe qui n'a pas terminé peut démontrer les exigences réalisées; seuls les critères démontrés sont crédités. Aucune reprise n'est accordée pour un travail inachevé.

9. Règles et politiques applicables

- **Usage autorisé :** les techniques offensives ne s'appliquent qu'aux programmes fournis dans l'environnement isolé du cours. Toute action contre un système tiers est interdite et traitée comme une inconduite, sans préjudice des recours prévus par la loi. Les shellcodes et exploits ne sont ni diffusés ni exécutés hors de l'environnement du cours.
- **Intégrité académique (règlement A-4) :** les rapports, démonstrations et programmes reflètent le travail de l'équipe; toute source réutilisée (SEED Labs, manuel, dépôts publics) est citée. La collaboration entre équipes se limite aux échanges d'idées; aucun partage de code ni de scripts.
- **Retards :** note de zéro pour tout travail remis après l'échéance, sauf prolongation autorisée ou accommodement approuvé.
- **Isolement :** l'isolement des machines virtuelles et des conteneurs utilisés pour les attaques doit être maintenu en tout temps; les contre-mesures ne sont modifiées que dans l'environnement isolé.
- **Accommodements :** communiquer avec le Service d'accommodements scolaires et informer le professeur suffisamment tôt.
- **Communication :** annonces, énoncés et remises sur Brightspace; courriel uOttawa pour toute question à l'équipe enseignante.

10. Grille d'évaluation détaillée (100 points = 5 % de la note finale)

La note du laboratoire se compose de quatre parties. La partie A est attribuée pendant la démonstration; les parties B, C et D sont attribuées à la correction du rapport et du dépôt. Les niveaux « maîtrisé », « satisfaisant », « partiel » et « non démontré » correspondent respectivement à 100 %, 70 %, 40 % et 0 % des points du critère; le correcteur peut attribuer une valeur intermédiaire justifiée par un commentaire.

Partie	Objet	Points
A	Démonstration devant l'assistant (présence de tous les membres obligatoire; note individualisable)	30
B	Attaques mémoire et outils de bas niveau (dont la composante L2-B)	30
C	Défense par le typage, propriétés et code	25
D	Rapport, traces, corpus et travail d'équipe	15
	Total	100

A. Démonstration — 30 points

Critère	Pts	Maîtrisé (100 %)	Satisfaisant (70 %)	Partiel (40 %)	Non démontré (0)
A1 Shellcode et débordement en direct (E1–E3, T1–T3)	7	Disposition et distance montrées; shellcode sans octet nul; débordement reproduit en direct avec privilège (Niveaux 1–3); cause première énoncée.	Exploitation reproduite avec aide; explication hésitante.	Exploitation instable ou limitée au Niveau 1.	Aucune exploitation.
A2 Contre-mesures (E4, T4)	5	dash, ASLR, StackGuard et NX traités; effet et limites de chacun montrés et expliqués.	Effet montré, explication approximative.	Une ou deux contre-mesures seulement.	Non démontré.
A3 Return-to-libc (E5, T5)	6	Pile non exécutable contournée par system(); offsets X, Y, Z et adresses expliqués.	Attaque reproduite avec aide; cause approximative.	Tentative partielle.	Non démontré.
A4 Exploitation à distance (E9, T8)	4	Serveur en conteneur exploité à distance par TCP; différences avec l'attaque locale expliquées.	Exploitation reproduite avec aide.	Conteneurs en place, exploitation partielle.	Non démontré.
A5 Défense Ada (E7, T7)	5	Même entrée désormais inerte : Constraint_Error levé et traité; mécanisme expliqué.	Exception montrée, une vérification manquante.	Comportement partiel ou non joué.	Non démontré.
A6 Questions individuelles (chaque membre)	3	Chaque membre explique la pile, le shellcode, le return-to-libc et le contrôle de contrainte, et effectue la modification demandée.	Réponses correctes avec aide; modification partielle.	Réponses vagues; un membre ne connaît qu'une partie.	Un membre ne peut expliquer le travail (règlement A-4).

B. Attaques mémoire et outils de bas niveau — 30 points

Critère	Pts	Maîtrisé (100 %)	Satisfaisant (70 %)	Partiel (40 %)	Non démontré (0)
B1 Disposition de la pile et contre-mesures (E1, E4)	7	Cadre d'appel et distance exacts; effet de dash, ASLR, StackGuard et NX documenté et expliqué, avec leurs limites.	Disposition correcte, analyse des contre-mesures incomplète.	Une partie seulement, ou mal expliquée.	Absente ou erronée.
B2 Exploitations reproductibles (E2, E3, E5, E9)	13	Shellcode, débordement (Niveaux 1–3), return-to-libc et exploitation à distance (Serveur) documentés, scripts rejouables, causes premières et propriétés violées énoncées.	Trois des quatre, une cause approximative.	Une ou deux exploitations reproductibles.	Aucune exploitation reproductible.

Critère	Pts	Maîtrisé (100 %)	Satisfaisant (70 %)	Partiel (40 %)	Non démontré (0)
B3 Chaîne source–assembleur–code machine (E6, L2-B, CLO 9)	10	Source, assembleur et code machine reliés; prologue/épilogue/ret identifiés; effet d'au moins deux options analysés et limites exposées.	Chaîne établie, analyse d'une seule option ou limites faibles.	Correspondance partielle ou sans analyse des options.	Absente.

C. Défense par le typage, propriétés et code — 25 points

Critère	Pts	Maîtrisé (100 %)	Satisfaisant (70 %)	Partiel (40 %)	Non démontré (0)
C1 Réécriture en Ada et contrôle de contrainte (E7)	10	Routine réécrite avec sous-types contraints; Constraint_Error levé et traité dans un état sûr; entrée de E3 devenue inerte.	Réécriture correcte, gestion de l'exception partielle.	Réécriture incomplète; débordement encore possible.	Routine absente ou non fonctionnelle.
C2 Propriétés vérifiables (E8)	8	Hypothèses violées écrites comme assertions ou propriétés claires (longueur, borne, invariant), prêtes pour SPARK.	Propriétés écrites, une ou deux imprécises.	Propriétés vagues.	Propriétés absentes.
C3 Qualité du code et des scripts	7	Code et scripts lisibles, commentés, compilant sans avertissement; réglages d'attaque explicites et reproductibles.	Code correct avec quelques avertissements.	Code peu commenté ou peu reproductible.	Code non fonctionnel.

D. Rapport, traces, corpus et travail d'équipe — 15 points

Critère	Pts	Maîtrisé (100 %)	Satisfaisant (70 %)	Partiel (40 %)	Non démontré (0)
D1 Rapport et tableau des essais (T1–T8)	6	Structure complète; chaque essai avec procédure, résultat, trace et verdict; limites discutées.	Rapport complet, preuves manquantes pour un ou deux essais.	Rapport partiel; moins de la moitié des essais tracés.	Rapport absent ou sans traces.
D2 Corpus d'entrées mémoire légué	4	Au moins huit entrées documentées et rejouables, rangées pour le projet final.	Corpus présent, descriptions sommaires.	Corpus incomplet (moins de huit entrées).	Aucun corpus.
D3 Dépôt Git et journal	5	Contributions substantielles et identifiables des deux membres; rotation des rôles inversée et consignée; JOURNAL.md tenu à chaque séance.	Journal présent, contribution d'un membre peu visible.	Journal partiel.	Aucun dépôt ni journal.

Pénalités et règles de calcul

Situation	Conséquence
Remise après le 16 octobre 2026, 23 h 59	Zéro pour le laboratoire (sauf prolongation autorisée ou accommodement approuvé).
Membre absent à la démonstration sans justificatif accepté	Zéro pour la partie A (30 points) pour ce membre; le membre présent est évalué normalement.
Action contre un système hors de l'environnement du cours	Traitée comme une inconduite (plan de cours, « Usage autorisé »); note du laboratoire compromise et signalement selon le règlement.
Attaque ou défense non reproductible à partir du dépôt	Le critère correspondant est limité à ce qui est démontré à la séance 2; les scripts non rejouables ne sont pas crédités.
Travail non expliqué par un membre à la démonstration	Critère A6 à zéro pour ce membre et note de la partie A réduite de moitié pour ce membre; renvoi au règlement A-4 si le travail n'est manifestement pas le sien.
Usage non déclaré d'outils d'IA générative ou sources non citées	Traité comme une remise non conforme au sens du règlement A-4.

Le corpus d'entrées mémoire (D2) et les propriétés violées (E8) alimentent le banc d'essai et les contrats du projet final (laboratoire 5), et sont réécrits en assertions vérifiables au laboratoire 3. La composante L2-B (CLO 9) est consignée distinctement au tableau de traçabilité du plan de cours.

11. Références

Les références ci-dessous suffisent au laboratoire 2. La liste complète des manuels, de la documentation des outils et des ressources figure à la section « Manuels du cours et références » du plan de cours.

Lecture principale

- Wenliang Du. Computer & Internet Security: A Hands-on Approach, 3e édition, 2022. ISBN : 978-1-7330039-4-0. Manuel du cours. Préparation du laboratoire 2 : chapitre 4 (débordement de tampon), chapitre 5 (attaque return-to-libc) et chapitre 9 (développement de shellcode).
- AdaCore. Introduction to Ada (version 2026-08). Chapitres « Strongly typed language », « Records » et « Arrays » pour la réécriture avec sous-types contraints (partie D). Fichier Ada.pdf sur Brightspace.

Ressources de laboratoire

- SEED Labs. Énoncés, machine virtuelle et conteneurs à télécharger à l'adresse <https://seedsecuritylabs.org/index.html>. Utiliser la suite SEED 2.0. Ce laboratoire s'appuie sur les thèmes « Shellcode Development Lab », « Buffer Overflow Attack Lab (Set-UID Version) » et « Return-to-libc Attack Lab »; l'extension obligatoire utilise « Buffer Overflow Attack Lab (Server Version) ». Les dossiers « Labsetup » sont fournis dans le dépôt.
- Code d'accompagnement du manuel (SEED Book), dépôt public de W. Du, fourni dans Books/BookCode-master (Buffer_Overflow, Return_to_Libc, Shellcode) : exemples commentés utiles à la préparation, p. ex. envaddr.c (adresse d'une variable d'environnement), generic_shellcode_32/64.s et convert.py.
- Documentation des outils. Pages de manuel Linux de gcc, gdb, objdump, readelf, nasm, ld, execstack et sysctl (kernel.randomize_va_space), ainsi que de gnatmake et gprbuild.

Normes et bonnes pratiques (facultatif)

- SEI CERT C Coding Standard. Règles de codage sécurisé en C, notamment la gestion des tableaux et des chaînes (ARR, STR) et de la mémoire (MEM).
- OWASP et NIST. OWASP Top 10 et NIST Cybersecurity Framework 2.0, pour situer les vulnérabilités mémoire et le codage sécurisé dans un cadre plus large.

Les énoncés et documents de cours sont diffusés sur Brightspace; l'environnement SEED se télécharge sur le site officiel. Les techniques de ce laboratoire ne s'appliquent qu'à l'environnement isolé du cours.